Mécanismes de sécurité de la base de données - pili-pos-android

Plateforme : Android 13 (API 33), terminal Sunmi D3 Pro (variante avec services Google)

Conformément au §II.C du Manuel de certification SCE 2.0, ce document décrit les **mécanismes techniques et organisationnels** qui protègent la base de données fiscale contre la modification, la suppression et l'accès non autorisé.

1. Mécanismes au niveau SQLite

1.1 WAL (Write-Ahead Logging)

```
PRAGMA journal_mode = WAL;
```

Activé au démarrage par `pos_db::migrate::run_migrations` (`crates/pos-db/src/migrate.rs`, lignes 162 à 166 : `journal_mode=WAL`, `synchronous=FULL`, `foreign_keys=ON`).

Le mode WAL garantit : - **Atomicité crash-safe** : un événement fiscal n'est commité que lorsque toute la transaction (compteur + archive + clôture commande) a été inscrite dans le WAL et synchronisée sur disque. Le même démarrage fixe `PRAGMA synchronous = FULL` (test `commits_are_synchronous_full` dans `migrate.rs`) : chaque commit atteint le disque avant de retourner, y compris en mode WAL. La requête de signature est donc durable avant d'être envoyée au FDM. - **Cohérence en cas de coupure de courant** : SQLite rejoue ou abandonne automatiquement les transactions partielles au prochain démarrage. Aucun événement n'est jamais à moitié écrit.

1.2 Foreign keys

```
PRAGMA foreign_keys = ON;
```

Activé au démarrage. Empêche toute insertion `order_lines.order_id` ou `fiscal_events.order_id` qui pointerait sur une commande inexistante.

1.3 Contraintes CHECK

Au niveau colonne : - `fiscal_events.event_type IN ('N','P','T','C','I','F','S','R')` - rejette toute insertion avec un label invalide. - `orders.status IN ('open','closed','voided')` - rejette tout statut inconnu. - `orders.order_type IN ('dine_in','takeaway')` - rejette tout type inconnu. - `menu_items.vat_code IN ('A','B','C','D','X')` - rejette tout code TVA invalide. - `payments.method IN ('cash','card','meal_voucher','other')` (migration 8) - restreint les modes de paiement. - `users.role IN ('admin','cashier','waiter')` - restreint les rôles à ceux mappés sur les profils SCE 2.0. - `peppol_deliveries.status IN ('queued','accepted','failed')` et `peppol_deliveries.lookup_status`, `b2b_customers.peppol_status IN ('unknown','registered','not_register` (migration 40) - restreignent les statuts de livraison et de recherche Peppol. - `sold_out_log.action IN ('set','clear','expired')` (migration 41), `menu_change_log.entity_type` et `menu_change_log.action IN ('create','update')` (migration 42).

1.4 Contraintes d'unicité

- `fiscal_events UNIQUE(event_type, sequence_number)` - empêche la double insertion du même couple (type, compteur). Couplée à la monotonie du compteur, garantit la **stricte séquentialité** exigée par la spec SCE 2.0.
- `users` primary key sur `insz` - un même INSZ ne peut pas être enregistré deux fois.
- `translations UNIQUE(entity_type, entity_id, field, lang)` - une seule traduction par couple entité / champ / langue.

2. Mécanismes au niveau applicatif

2.1 Append-only pour `fiscal_events`

Deux protections indépendantes, la seconde ne dépendant pas du code.

Triggers SQLite (niveau base). Le schéma refuse la modification et la suppression, même par un accès direct à la base :

- `fiscal_events_no_delete` et `fiscal_events_no_rewrite` (migration 4) : tout `DELETE` et tout `UPDATE` sur `fiscal_events` avorte (`fiscal events are append-only`) ;
- `z_reports_no_delete` et `z_reports_no_rewrite` (migration 4), même règle pour les rapports Z ;
- `fiscal_events_period_open` (migration 10) : un `INSERT` dont le `bookingPeriodId` désigne une période déjà close avorte ;
- `order_entries_no_delete` et `order_entries_no_rewrite` (migration 2), seul `sent_event_id` reste modifiable ;
- `booking_periods_no_reopen` et `booking_periods_no_delete` (migration 3) ;
- `pending_signs_request_immutable`, `pending_signs_context_on_resolution`, `pending_signs_no_reopen` et `pending_signs_no_delete` (migration 10) ;
- `till_discounts_no_delete`, `till_discounts_no_update` et `order_entries_till_op_fixed` (migration 30 : remises de la caisse ; la marque `till_op` d'une ligne du journal ne change jamais) ;
- `invoices_*`, `invoice_sales_*` et `peppol_delivery_log_*` (migration 40 : factures émises et tentatives d'envoi Peppol, jamais de `UPDATE` ni de `DELETE`), `sold_out_log_*` (migration 41) et `menu_change_log_*` (migration 42) ;
- `ticket_deliveries_*`, `original_prints_*`, `order_price_changes_*` et `integration_switch_log_*` (liste complète dans `SCHEMA_ER_FR.md` §4).

Revue de code (niveau applicatif). Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_events` ni `UPDATE fiscal_events SET ...` (vérification grep - cf. `DECLARATION_FONCTIONS_INTERDITES_FR.md` §1 et §2). Les seules opérations possibles sont :

- `INSERT` via `pos_fiscal::archive::archive_event` (`crates/pos-fiscal/src/archive.rs:38`).
- `SELECT` via les fonctions de `pos_db::queries::fiscal`.

Tout patch introduisant un DELETE/UPDATE sur `fiscal_events` est rejeté en pull request (politique organisationnelle, cf. `PROFIL_ENTREPRISE_FR.md`). Les tests `archived_event_cannot_be_deleted` et `archived_event_cannot_be_rewritten` (`crates/pos-fiscal/src/archive.rs`) vérifient les triggers.

2.2 Transaction atomique compteur + archive

Chaque événement fiscal est signé sous un verrou unique, puis inséré dans une transaction SQLite qui inclut :

1. L'allocation du compteur (`FiscalCounter::next` - incrément persistant).
2. L'insertion de la ligne `fiscal_events`.
3. (Le cas échéant) la fermeture de la commande (`UPDATE orders SET status = 'closed'`).
4. La clôture de la ligne `pending_signs` de la requête (statut `signed`).

Si l'une de ces opérations échoue, **toute la transaction est annulée** (rollback) : ni le compteur ni l'archive ni le statut de commande ne sont modifiés, et la ligne `pending_signs` reste ouverte : l'opération suivante renvoie la requête à l'identique et archive l'événement (aucun événement signé n'est perdu). Le déclencheur `fiscal_events_period_open` refuse l'archivage dans une période déjà fermée. Voir `STRATEGIE_COMPTEURS_FR.md` §5 et la régression `pili-pos::spf_e2e_no_gap`.

2.3 Aucune suppression de compteur

Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_counters` ni `UPDATE fiscal_counters SET current_value = ...` ailleurs que dans `FiscalCounter::next` (qui incrémente exclusivement, jamais ne diminue). Voir `DECLARATION_FONCTIONS_INTERDITES_FR.md` §4.

2.4 Validation des entrées

- INSZ validé par `pos_core::user::validate_insz` (11 chiffres ASCII + mod-97) avant tout appel commande mutant un événement fiscal.
- Montants exprimés en **centimes en `i64`** - pas de flottants pour les calculs monétaires. Toute conversion depuis l'interface utilisateur passe par un parsing strict (panique si overflow ou caractères invalides).
- Horloge système **non modifiable** par l'opérateur (restriction du propriétaire de l'appareil, pas d'accès root, heure du réseau activée, fuseau Europe/Brussels imposé).

2.5 Badges du personnel (table `staff_badges` et journaux)

Option désactivée par défaut (`badge_settings.reader = 'off'`), mais les quatre tables existent sur toute installation (migration 70 `staff-badges`) :

Table	Rôle	Protections au niveau SQLite
`badge_settings`	Réglage unique (ligne `id = 1`) : lecteur (`off`, `nfc` ou `usb`), connexion par badge, réglages du lecteur USB, empreinte de contrôle de la clé	CHECK sur le lecteur, l'interrupteur et chaque borne (écart 10 à 200 ms, longueurs 4 à 32 et 4 à 64, minimum inférieur ou égal au maximum)
`staff_badges`	Un lien badge / employé : empreinte (64 caractères hexadécimaux), INSZ, qui l'a lié et quand, retrait (quand, par qui, motif `manager`, `deactivated` ou `key_changed`)	Deux index uniques partiels : une seule empreinte active, un seul badge actif par employé. Déclencheur `staff_badges_no_delete` : aucune suppression. Déclencheur `staff_badges_revoke_only` : la seule modification possible d'une ligne est son retrait, une fois ; l'empreinte, l'INSZ et les dates de lien ne changent jamais
`badge_log`	Journal des liens, retraits et changements de réglage (qui, quand, détail)	Ajout seul : déclencheurs contre toute modification et toute suppression
`badge_scans`	Journal des passages (heure, INSZ, résultat, référence de 12 caractères de l'empreinte, texte du refus ou du FDM)	Ajout seul, comme `badge_log`

- **Aucun numéro de badge en base.** Le numéro lu est normalisé puis transformé en empreinte `HMAC-SHA256(clé, "pili-badge/1" + numéro)` avant toute écriture ; ni la base (et son WAL), ni les journaux applicatifs, ni un événement fiscal ne contiennent le numéro (un test lit les octets du fichier de la base après un lien et un passage pour le vérifier). Les journaux ne portent que la référence de 12 caractères.
- **La clé n'est pas dans la base.** Elle fait 32 octets tirés au hasard (`getrandom`), est créée une seule fois sur l'appareil, sous le nom `badge.key` dans `files/engine/` (mode 0600, écrite sous un nom provisoire puis liée à sa place, jamais remplacée) et **ne quitte jamais l'appareil** : elle n'est pas dans `pos.db`, donc ni dans les sauvegardes (`backups/`), ni dans les copies de la base, ni dans l'export nocturne, ni dans l'export pour le SPF. Sans la clé, une base volée ne permet pas de retrouver un numéro de badge, même en énumérant les numéros possibles (4 à 7 octets seulement).
- **Clé perdue.** `badge_settings.key_check` garde les 16 premiers caractères hexadécimaux de `HMAC(clé, "pili-badge-check/1")`. À l'ouverture du moteur, et avant chaque passage ou lien, si le fichier de la clé manque (restauration d'une sauvegarde sur un autre matériel) ou donne un autre contrôle, tous les badges actifs sont retirés avec le motif `key_changed` (une ligne de `badge_log` chacun) ; un fichier qui existe mais ne se lit pas n'est pas une clé perdue : les commandes répondent `badge_key_unavailable` et rien n'est retiré.

- **Un Gérant n'est jamais ouvert par un badge seul** : son badge demande son PIN (`pin_required`), si bien que le vol d'un badge de Gérant ne donne accès à aucune action de Gérant.
- Le badge ne remplace jamais l'INSZ : l'événement S reste signé par le FDM avec l'INSZ de l'employé.

3. Mécanismes au niveau système

3.1 Stockage privé de l'application

Toutes les données vivent dans le répertoire privé de l'application (`files/`), que le bac à sable d'Android réserve à l'identifiant d'utilisateur (UID) unique de `pili-pos-android`. Aucune autre application ne les lit ; l'application ne demande aucune permission de stockage et son manifeste n'a pas d'`android:installLocation`, donc Android ne les déplace jamais vers une carte SD (`PLATEFORME_ANDROID_FR.md` §5).

Élément	Répertoire sous `files/`	Fichiers et remarque
Base fiscale	`engine/`	`pos.db`, `pos.db-wal`, `pos.db-shm` (annexes WAL du §1.1)
Verrou d'instance	`engine/`	`engine.lock`, fichier vide, sans donnée : une seule instance écrit dans `pos.db`
Clé des badges	`engine/`	`badge.key`, 32 octets aléatoires, mode 0600, hors de `pos.db` et donc des sauvegardes (§2.5)
Sauvegardes	`engine/backups/`	`pos-*.db`, voir §5
Exports nocturnes locaux	`engine/exports/`	Facultatif, voir §5.1
Journaux applicatifs	`logs/`	Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §1.3
Configuration	`provisioning/bundle/`	`pos.json`, dans le lot de configuration signé, vérifié à l'import
Journal de la plateforme	`platform/`	`audit.jsonl`, voir §7

Aucun de ces fichiers n'est modifiable par l'opérateur : l'interface n'offre aucun écran de fichiers et la configuration ne change que par l'import d'un nouveau lot signé (`PLATEFORME_ANDROID_FR.md` §3).

3.2 Propriétaire de l'appareil

L'application est le **propriétaire de l'appareil** (device owner) du terminal. Elle est installée en tant que tel au banc du producteur (`dpm set-device-owner`) ou, sur site après une réinitialisation d'usine, par un provisionnement par code QR, dans les deux cas sans aucun compte Google. Le propriétaire de l'appareil impose, entre autres : pas de réinitialisation d'usine ni de démarrage en mode sans échec, pas de désinstallation de l'application, pas d'installation depuis une source inconnue, pas de nouvel utilisateur ni de compte, pas de transfert de fichiers par USB, pas de changement de date ni d'heure (liste complète : `PLATEFORME_ANDROID_FR.md` §3). Les restrictions des cartes amovibles et du Wi-Fi sont volontairement laissées libres : la carte SD ou la clé sert à l'export SPF (§8) et le Wi-Fi se prépare au banc avant l'import du lot (`MANUEL_INSTALLATION_FR.md` §4.3).

Les mesures propres à la variante avec services Google sont construites (Play Store et applications Google masqués, `DISALLOW_INSTALL_APPS` levée pour la seule session de l'outil de mise à jour, politique de mise à jour système à fenêtre de nuit de 03:00 à 05:00 : `PLATEFORME_ANDROID_FR.md` §3.7). Leur mesure sur le profil émulé avec services Google (15.10.2026, EV-13, `evidence/gms/EV13_2026-10-15.md`) : onze paquets Google masqués, restrictions d'installation posées, fenêtre de mise à jour de 03:00 à 05:00. Play Protect sur un vrai terminal : À effectuer (EV-15 : D3 Pro réel).

3.3 Environnement verrouillé

À la place de l'image scellée de la version Linux, le terminal est verrouillé par l'application elle-même (détails et limites : PLATEFORME_ANDROID_FR.md §3, à ne pas sur-vendre ici) :

- l'application est l'écran d'accueil et tourne en mode Lock Task (kiosque) ; écran de verrouillage et barre d'état sont désactivés ;
- aucun port n'écoute : toutes les connexions sont sortantes (PLATEFORME_ANDROID_FR.md §6) ;
- **pas d'assistance à distance** : aucun programme de prise de main n'existe sur Android, le support se fait sur site (PLATEFORME_ANDROID_FR.md §7) ;
- l'image système est celle du constructeur, que le producteur ne reconstruit pas : ses surfaces propres (réglages, applications du constructeur) ne sont vérifiées que sur le profil émulé. Vérification sur le matériel réel : À effectuer (EV-15 : terminal D3 Pro réel).

3.4 Pas d'élévation de privilèges

- Aucun accès `root`, aucun `su`, aucun shell n'est fourni à l'opérateur ni au contrôleur : le terminal n'a ni console, ni compte de contrôle, ni SSH.
- Dans une version de production, le débogage USB (adb) est coupé par le propriétaire de l'appareil dès que le lot de configuration est installé. Les versions de développement (suffixe `–dev`) le laissent actif et ne sont jamais déclarées.
- Les restrictions `no_factory_reset` et `no_safe_boot` sont posées par le propriétaire de l'appareil (§3.2). Leur effet sur le D3 Pro réel (combinaison de touches du mode sans échec, menu recovery, ce que ce menu permet d'effacer ou d'installer) n'est pas démontré sur l'appareil émulé : À effectuer (EV-15 : combinaison de touches du mode sans échec et menu recovery du D3 Pro réel).

3.5 Démarrage

L'application redémarre seule : à la mise sous tension (récepteur de démarrage puis service de premier plan) et après un état fatal du moteur (chien de garde, trois redémarrages au plus en dix minutes, PLATE–FORME_ANDROID_FR.md §8). Le **Verified Boot** du système du constructeur n'est pas déclaré dans ce dossier : À effectuer (EV-15 : état du Verified Boot et du chargeur de démarrage du D3 Pro réel). Contrairement à la version Linux, aucune configuration de BIOS ni de mot de passe de démarrage n'est demandée au technicien.

3.6 Chiffrement du stockage

Android chiffre en général le stockage de données (chiffrement par fichier), mais ce dossier ne le déclare pas pour le D3 Pro réel : À effectuer (EV-15 : chiffrement du stockage du D3 Pro réel). Le producteur n'ajoute aucun chiffrement de `pos.db` et n'en revendique aucun. Ce que le dossier garantit sans lui : l'intégrité des données fiscales (triggers §2.1, signatures du FDM §4.1, journal du FDM et du SPF Finances §7), et le chiffrement de bout en bout de la seule copie qui quitte le terminal (§5.1). La copie de l'export SPF (§8) est, elle, en clair sur le support que choisit le Gérant, comme un contrôle l'exige.

4. Mécanismes au niveau cryptographique

4.1 Signature ECDSA secp256r1 (par le FDM)

Chaque événement fiscal porte la signature `digitalSignature` du FDM dans `fiscal_events.fdm_response_json`, conservée telle que le FDM l'a renvoyée, avec la réponse complète. Le FDM garde lui-même chaque événement signé et le transmet au SPF Finances. Une ligne `fiscal_events` modifiée a posteriori (par exemple via un accès physique) ne correspondrait donc plus à l'événement signé détenu par le FDM et le SPF Finances : la comparaison révèle la falsification.

Le POS ne détient pas la clé publique du FDM (le schéma GraphQL du FDM ne l'expose pas) et ne vérifie donc pas lui-même la signature ECDSA. Il contrôle en revanche, avant tout archivage, que chaque réponse correspond à la requête envoyée (`crates/pos-fiscal/src/response_check.rs`) :

- `posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId` et `terminalId` identiques à la requête ;

- `eventOperation` et `eventLabel` conformes à la mutation appelée (label T en mode formation) ;
- `digitalSignature` présente et en base64 valide ;
- pour un événement N : `shortSignature` présente et égale à Hex(SHA1(Base64Decode(digitalSignature))), `verificationUrl` présente.

Une réponse qui ne respecte pas ces règles n'est jamais archivée : la vente est refusée, rien n'est imprimé, la commande reste ouverte et l'écran affiche « Réponse de la boîte noire incohérente ». Un compteur FDM qui ne progresse pas (même FDM) est consigné dans le journal technique.

4.2 Empreintes de l'APK

Les empreintes SHA-1 de l'APK signé `pili-pos-android` 1.0.0 et de la bibliothèque `libpos_ffi.so` qu'il embarque (le moteur) sont consignées dans le formulaire de demande de certification : À effectuer (EV-01 : SHA-1 et SHA-256 de l'APK ; EV-02 : SHA-1 de `libpos_ffi.so`). L'écran « À propos » les affiche avec l'empreinte SHA-256 du certificat de signature (EV-03), et `identity.json` de l'export SPF (§8) les reprend. Sur un terminal de production, le débogage USB est coupé (§3.4) : le fichier `base.apk` exécuté ne peut pas en être extrait. Le contrôleur compare donc les valeurs affichées par le terminal (écran « À propos » et `identity.json`) avec les valeurs consignées ; ces valeurs sont fournies par l'application elle-même et ne constituent pas une vérification indépendante de l'application exécutée. Les vérifications indépendantes portent sur l'artefact public déclaré (`shasum -a 1` de l'APK publié) et sur le build reproductible (EV-01). Détail et reproductibilité du build : `GARANTIE_MAJ_LANGUES_FR.md` §1.2.

4.3 Hash du code source

Tag git de la révision certifiée (`android-v1.0.0`) et tag de gel du moteur commun `linux-v1.0.0-rc2` (À effectuer (EV-04 : commit du tag)). Permet de reproduire l'APK à l'identique et de vérifier l'absence de modifications post-certification.

5. Sauvegardes

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §1.2.

Politique : - Sauvegarde automatique après chaque Z-report, juste après son archivage (`signing::follow_up` pour un Z, qui appelle `commands::report::backup_database_impl`, via SQLite Online Backup API). Un échec de sauvegarde est journalisé et n'annule pas le Z. - Fichier daté à la seconde, dans `files/engine/backups/`, jamais écrasé. - La clé des badges (§2.5) n'est **pas** sauvegardée avec la base : restaurer une sauvegarde sur un autre appareil retire les badges (motif `key_changed`), à lier de nouveau. - **Rétention : les 7 sauvegardes les plus récentes.** Après chaque Z, le moteur garde les 7 copies les plus récentes et supprime les plus anciennes (décision du producteur du 2026-10-10 ; le commit de ce changement du moteur dans l'étiquette de gel : À effectuer (EV-19 : commit du changement du moteur)). Chaque copie est une copie complète de `pos.db` : le journal fiscal (`fiscal_events`), qui est dans `pos.db`, n'est jamais supprimé et contient tout ce que contenaient les copies supprimées. Sans cette règle, 365 copies d'une base qui grossit rempliraient la mémoire interne en quelques mois. Le moteur étant commun, la règle vaut aussi pour la version Linux au même tag de gel. - Stockage local, dans la mémoire interne uniquement. Aucune sauvegarde dans le nuage ni transfert d'appareil : `allowBackup="false"` et `android/app/src/main/res/xml/data_extraction_rules.xml`, qui exclut tous les domaines de `cloud-backup` et de `device-transfer`. La seule copie distante prévue est l'export nocturne facultatif et chiffré du §5.1. - Pour un contrôle, l'export SPF (§8) copie `pos.db` et les 7 sauvegardes conservées.

5.1 Sauvegarde chiffrée de bout en bout (export nocturne, facultatif)

Seulement si la section `export` de `pos.json` est renseignée (`EXPORT_NOCTURNE_FR.md`). Elle s'ajoute aux sauvegardes locales ci-dessus, qu'elle ne remplace pas ni ne modifie.

- **Chiffrement de bout en bout.** La copie de `pos.db` est chiffrée sur la caisse avec age (X25519) pour la clé publique du propriétaire de l'établissement et, en option, du producteur, avant d'être envoyée. Les

clés privées ne sont jamais sur la caisse ni sur le serveur de réception, qui ne détient la sauvegarde que sous forme chiffrée et ne peut pas la lire (il reçoit aussi la copie statistique et le manifeste, non chiffrés).

- **Aucun texte clair conservé.** La copie intermédiaire de la base n'existe que le temps du chiffrement, dans un répertoire de travail vidé au démarrage et avant chaque préparation ; elle est supprimée aussi en cas d'échec. Seule la copie statistique (sans donnée directement identifiante, opérateurs sous pseudonyme) est conservée en clair, avec le fichier chiffré.
- **Intégrité.** Chaque fichier est envoyé avec son empreinte SHA-256, que le récepteur vérifie ; le manifeste, envoyé en dernier, répète les empreintes et les tailles et marque l'export complet.
- **Transport.** HTTPS vers le point de réception propre à l'installation ; jeton porteur dans un fichier séparé de `pos.json`, jamais journalisé.
- **Aucune écriture dans la base.** La préparation lit `pos.db` par une connexion en lecture seule, hors du verrou de signature : elle ne peut ni modifier `fiscal_events` ni retarder une vente ou un Z.
- **Stockage local** dans la mémoire interne, à côté de `pos.db` (`files/engine/exports/`, protégée comme elle par le bac à sable de l'application, §3.1), jamais sur une carte SD.

6. Confidentialité (GDPR)

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §6 et `PROFIL_ENTREPRISE_FR.md` §GDPR.

Données de l'équipe :

- INSZ stocké en clair (donnée personnelle obligatoire fiscalement).
- PIN stocké en hash bcrypt (jamais en clair).
- Badges (option) : jamais le numéro du badge, seulement une empreinte HMAC-SHA256 avec une clé propre à l'appareil, hors de la base, des sauvegardes et des exports (§2.5) ; usage limité à la présence et à la connexion ; aucune donnée biométrique ; le badge est facultatif pour l'employé.
- Aucun INSZ n'est écrit dans les journaux applicatifs.

Données de clients. Le binaire certifié contient l'intégration HubRise (désactivée par défaut, activée par installation : `COMMANDES_EN_LIGNE_FR.md` §2). Dès qu'elle est activée, le POS traite des données de clients, ce qu'il ne faisait pas sans elle :

- **Commandes en ligne.** Chaque commande reçue est conservée telle que HubRise l'envoie dans `orders.raw_hubrise_json`, avec les données du client (nom, téléphone, adresse de livraison, notes), et dans `hubrise_events.payload_json`. Le ticket client non fiscal imprime le nom, le téléphone et l'adresse. Aucune purge automatique de ces colonnes n'existe : elles suivent la conservation de la base.
- **Ventes de la caisse renvoyées à HubRise.** Quand l'intégration est activée, chaque vente de la caisse est mise en file dans `hubrise_outbox` puis envoyée à HubRise (lignes, totaux, paiements ; pas de données de client).
- **Ticket numérique par e-mail.** L'adresse saisie est conservée en clair dans `ticket_deliveries.recipient` le temps de l'envoi, puis effacée (envoi réussi, repli papier, ou au plus tard 72 h après) ; une forme masquée reste. Le ticket et l'adresse du destinataire sont transmis à Scaleway Transactional Email pour l'envoi.
- **Facture B2B.** Le nom, l'adresse et le numéro de TVA du client professionnel sont conservés dans `b2b_customers` (table de travail, modifiable) et, figés, dans `invoices.customer_json` et dans le document UBL `peppol_deliveries.ubl_xml` (ajout seul, conservés comme le journal fiscal). Ils ne sont transmis à un tiers que pour un client belge et si l'installation a une section `peppol` : le document est alors envoyé au point d'accès Peppol (Storecove). Aucune donnée de particulier : une facture est établie pour un numéro de TVA contrôlé.

Jeton HubRise. `hubrise_connection.access_token` (jeton d'accès OAuth au compte HubRise de l'établissement) est stocké **en clair** dans `pos.db`. Il est donc présent dans chaque sauvegarde de la base et dans la copie de `pos.db` de l'export SPF (§8). Il est protégé comme le reste de la base : bac à sable de l'application (§3.1) ; chiffrement du stockage, §3.6. Il est supprimé à la déconnexion de HubRise.

Export : par défaut, le POS n'exporte automatiquement aucune donnée fiscale ; les seuls flux sortants sont ceux ci-dessus (HubRise et Scaleway, selon la configuration de l'installation), la communication avec le FDM et, si la section `export` de `pos.json` est renseignée, l'export nocturne (§5.1) : une sauvegarde chiffrée de

bout en bout, que le serveur de réception ne peut pas lire, et une copie statistique sans donnée directement identifiante (EXPORT_NOCTURNE_FR.md).

7. Audit trail croisé

Chaque événement fiscal est tracé à TROIS endroits indépendants :

1. **Côté POS** : table fiscal_events.
2. **Côté FDM** : journal interne du FDM (signe et stocke indépendamment).
3. **Côté SPF Finances** : remontée cloud quotidienne du FDM via le protocole FPSFIN_API_PROTOCOL_FDM_FINCLOUD_1.

Le contrôleur peut croiser ces trois sources pour repérer toute divergence. Une suppression locale de fiscal_events serait immédiatement visible par la non-correspondance avec le journal FDM.

Le terminal tient en plus un journal propre à la plateforme, audit.jsonl, dans files/platform/ : une ligne JSON par fait (règles appliquées, redémarrages du chien de garde, exports, mises à jour), chaque ajout synchronisé sur disque, rotation à 1 Mio avec 5 fichiers conservés. Ce n'est pas le journal fiscal, il n'est envoyé nulle part et il fait partie de l'export SPF (§8).

8. Accès du SPF Finances aux données (Art. 16 et 102)

Texte de référence cité par les autres annexes. Sur Android, il n'existe qu'une voie d'accès aux données d'un terminal en service : l'export déclenché par le Gérant sur une clé USB ou une carte SD.

8.1 Droits de l'opérateur

L'opérateur n'a aucun accès direct aux données : ni console, ni explorateur de fichiers, ni compte de contrôle. Son seul droit est la **lecture, sous forme d'export** : l'export copie, il ne modifie ni ne supprime aucune donnée existante de pos.db (il ajoute seulement une copie fraîche aux sauvegardes du terminal), et les triggers du §2.1 refusent toute modification des événements fiscaux.

8.2 Export par le Gérant

Écran « Export pour le SPF » des Paramètres, réservé à un utilisateur Gérant connecté (rôle administrateur). L'export est refusé tant qu'une commande ou une vente à emporter est ouverte ou que le moteur n'est pas prêt ; chaque refus est journalisé. Étapes :

1. Insérer la clé USB ou la carte SD (format exFAT ; une clé en FAT32 refuse les fichiers de 4 Gio et l'écran le signale).
2. Au premier usage, choisir sur ce support, avec le sélecteur de dossiers d'Android (accès SAF), un dossier nommé PILI-EXPORT (la racine du support est refusée).
3. Appuyer sur « Exporter ». Chaque fichier est copié, puis relu sur le support et comparé par empreinte SHA-256 ; une différence interrompt l'export, sans rien supprimer.
4. L'écran « Export terminé » affiche le dossier, le nombre et la taille des fichiers et l'empreinte SHA-256 de MANIFEST.sha256.

Le dossier créé s'appelle pili-export-<date>-<heure>-<deviceId> (heure de Bruxelles) et contient :

Entrée	Contenu
pos.db	Copie cohérente et fraîche de la base (API Online Backup de SQLite, cohérente en WAL), qui s'ouvre telle quelle
backups/	Les 7 sauvegardes les plus récentes (§5), jamais davantage
backups.txt	Inventaire des autres sauvegardes du terminal : nom, taille, SHA-256
logs/	Les journaux applicatifs (§3.1)

Entrée	Contenu
audit/	Les fichiers du journal de la plateforme (§7), dont la ligne de démarrage de cet export
identity.json	Identité du terminal : version, empreintes de l'APK et de libpos_ffi.so, certificat de signature, deviceId, posId, terminalId
LISEZMOI.txt	En français : le rôle de chaque fichier, la façon de vérifier, la date et le deviceId
MANIFEST.sha256	Empreinte SHA-256 de chaque autre fichier, format sha256sum, écrit en dernier

Un MANIFEST.sha256 absent signifie un export incomplet.

8.3 Ce que fait le contrôleur sur place

1. Faire exporter le Gérant, puis brancher la clé ou la carte sur son ordinateur.
2. Vérifier MANIFEST.sha256 (shasum -a 256 -c MANIFEST.sha256, ou sha256sum -c sous Linux) : chaque ligne doit afficher OK, et l'empreinte du fichier lui-même doit être celle de l'écran « Export terminé ».
3. Vérifier sqlite3 pos.db "PRAGMA integrity_check;" : ok.
4. Comparer les empreintes de identity.json avec celles consignées dans le formulaire (§4.2) et lire le dossier audit/ pour les exports passés.
5. Interroger pos.db en lecture avec les requêtes de FEUILLE_ACQUISITION_DONNEES_FR.md §2.3 et recouper chaque événement avec le FDM et le SPF Finances (§4.1 et §7).

8.4 Aucune autre voie

Sur un terminal provisionné, il n'existe aucun autre chemin vers les données : pas de console ni de compte de contrôle (le compte spf de l'image d'admission Linux n'a pas d'équivalent), pas d'assistance à distance, pas de port à l'écoute, débogage USB coupé et transfert de fichiers USB interdit (§3.2 à §3.4 ; pour la réinitialisation d'usine et le mode sans échec, restrictions posées, effet sur le D3 Pro réel : À effectuer (EV-15 : effet des restrictions sur le D3 Pro réel), §3.4). L'export nocturne (§5.1) n'est pas une voie d'accès pour le SPF : sa sauvegarde est chiffrée pour le propriétaire de l'établissement.

Le comportement sur un vrai support (exFAT, retrait en cours d'écriture) et le déroulé complet sur le terminal : À effectuer (EV-15 : vraie clé et vraie carte ; EV-10 : déroulé de l'export SPF sur les AVD).